

Padrões de Resiliência

Idempotência, Retry e Exponential Backoff

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Idempotência

Definição:

Operação idempotente produz o mesmo resultado independente de quantas vezes for executada. Essencial em sistemas distribuídos.

Por que é necessário?

- SQS Standard: at-least-once delivery → mensagens podem ser duplicadas
- Lambda: pode ser invocado mais de uma vez para o mesmo evento
- Redes distribuídas: retries podem reenviar a mesma mensagem

Como implementar:

- Use IDs únicos por operação (idempotency key)
- Verifique se a operação já foi processada antes de executar
- Armazene o estado em DynamoDB com TTL para controle eficiente

Suporte nativo AWS:

- SQS FIFO: exactly-once com Message Deduplication ID (janela de 5 min)
- API Gateway: idempotency key via header para chamadas críticas

Retry com Exponential Backoff

Por que usar retry?

Falhas transitórias são comuns: throttling, timeouts momentâneos, indisponibilidade temporária.

Exponential Backoff — Progressão:

- Tentativa 1: aguardar 1 segundo antes de tentar novamente
- Tentativa 2: aguardar 2 segundos
- Tentativa 3: aguardar 4 segundos
- Tentativa N: aguardar $2^{(N-1)}$ segundos (até um cap máximo)

AWS SDK: implementa retry + exponential backoff automaticamente

Quando fazer retry:

- Sim: erros 5xx (server), ThrottlingException, ProvisionedThroughputExceededException
- **NAO: erros 4xx (client errors) — retry não vai resolver**

Jitter — Evitando Thundering Herd

O Problema sem Jitter:

Múltiplos clientes fazem retry ao mesmo tempo → pico de carga → mais falhas → mais retries → loop infinito.

Solução: adicionar aleatoriedade (jitter) ao intervalo de espera

Tipos de Jitter:

- Full Jitter: $\text{sleep} = \text{random}(0, \min(\text{cap}, \text{base} \times 2^{\text{attempt}}))$

 - Melhor distribuição de carga no tempo

- Equal Jitter: $\text{sleep} = \text{cap}/2 + \text{random}(0, \text{cap}/2)$

 - Equilíbrio entre previsibilidade e distribuição

AWS SDK: implementa full jitter automaticamente junto com backoff

Resultado prático:

- Retries distribuídos no tempo → sem pico → sistema se recupera naturalmente

Padrão Completo de Resiliência

DLQ como Safety Net Final

- Após N retries sem sucesso → mensagem vai para DLQ
- Evita perda de mensagens com falha permanente
- Permite análise, debug e reproprocessamento manual
- Configure CloudWatch Alarm na DLQ para alertar

Os 4 Padrões Combinados

1. Idempotência → operação safe to retry
2. Exponential Backoff → não sobrecarrega o destino
3. Jitter → distribui retries no tempo
4. DLQ → captura o que não pode ser resolvido

Boas Práticas:

- maxReceiveCount típico: 3 a 5 tentativas antes de enviar para DLQ
- Monitore a DLQ com CloudWatch Alarms + SNS para notificar o time
- Combine os 4 padrões: cada um resolve um problema diferente

Revisão: Padrões de Resiliência



1. Idempotência

- Mesma operação executada N vezes = mesmo resultado
- Use idempotency keys + verificação de estado (DynamoDB)

2. Exponential Backoff

- Intervalo cresce exponencialmente (2^n segundos)
- Retry: 5xx e throttling. NAO retry: 4xx (client errors)

3. Jitter

- Aleatoriedade no intervalo de retry
- Previne thundering herd (pico de retries simultâneos)

4. Padrão Completo

- Idempotência + Backoff + Jitter + DLQ = sistema resiliente

LEMBRE-SE: Se questão menciona '*ThrottlingException*' ou '*retry sem sobrecarregar*' > *exponential backoff + jitter!*